

TD

COLORATION DE GRAPHE

On considère dans la suite un graphe non orienté G connexe et sans boucle. On notera les sommets de ce graphe de 0 à $n-1$ et on le représentera par sa matrice d'adjacence.

Rappelons que le degré d'un sommet s et le nombre d'arêtes dont s est une extrémité.

Dans ce TD les graphes seront représentés par leur matrice d'adjacences.

On donne une liste de couleurs (représentés par des nombres entiers) : `couleurs = [1, 1, 2]` signifie qu'on a un graphe à 3 sommets, le premier et le second son de même couleur, pas le dernier.

Pour le TD on essaye d'avoir une règle commune : 1 rouge, 2 bleu, 3 noir, 4 vert, 5 ... On considère que la couleur 0 est pour les sommets non coloriés.

L'objectif de ce problème est d'attribuer une couleur à chaque sommet du graphe de façon à ce que deux sommets voisins soient de couleurs différentes.

Voici deux exemples, un bien coloré, un mal coloré.

```
1
2 Mat = [[0,1,0,0,1,1],[1,0,1,0,0,0],[0,1,0,0,1,0],[0,0,0,0,1,0],[1,0,1,1,0,0],[1,0,0,0,0,0]]
3 couleursbien = [1,2,1,3,4,2]
4 couleursmal = [1,2,2,3,4,2]
```

1) Dessiner les deux versions colorées de ce graphe.

On redonne la fonction voisins qui donne la liste des sommets voisins d'un sommet .

```
1 def Voisins(M,S) :
2     V =[]
3     for j in range(len(M)):
4         if M[S][j]==1:
5             V.append (j)
6     return V
```

1 L'ALGORITHME NAIF

La version naïve est simple :

- On parcourt les sommets s dans l'ordre
- On attribue au sommet s la plus petite couleur non utilisée par ses voisins

2) Compléter la fonction `*Naive(G)*` qui prend en argument la matrice d'adjacence d'un graphe G et le colorie.

```
1 def Naif(G):
2     n=len(G)
3     c =[0 for i in range(n)] # pas de couleurs au début
4     for s in range(n):
5         # On fait la liste des couleurs des voisins de S
6         CouleursVoisins = ....
7         # on donne la première couleur non attribuée dans les voisins
8         couleur = ...
9         while couleur in CouleursVoisins :
10             couleur += ...
11         c[s] = couleur
12     return c
```

3) Tester avec le graphe $G = [[0,0,1,1,0],[0,0,0,1,1],[1,0,0,1,1],[1,1,1,0,1],[0,1,1,1,0]]$.

4) Le nombre de couleurs utilisées est-il minimal?

2 L'ALGORITHME GROUTON

Il semble intéressant de commencer par le sommet ayant le degré le plus élevé, car ainsi il n'y aura qu'une seule couleur non disponible pour les nombreux sommets voisins de ce sommet, ce qui devrait laisser plus de choix pour la suite.

C'est cette approche gloutonne que l'on va mettre en place. Glouton car à une étape, on fait le choix d'un optimum local, mais est-ce qu'au final on aura un optimum global?

Commençons par trier tout ça, on utilise un tri à bulles (parce que pourquoi pas), qu'il faut donc comprendre et adapter.

5) S'inspirer du tri à bulles pour trier les degré dans l'ordre décroissant.

```

1  def TriBulles(L) :
2      n=len(L)
3      for i in range(n-1) :
4          for j in range(0,n-i-1) :
5              if L[j]< L[j+1] :
6                  L[j] ,L[j+1] = L[j +1], L[j]
7      return L
8
9  def TriDegre(G):
10     n =len(G)
11     # On créé la liste des degrés
12     d = [ len(Voisins(G,s)) for s in range(n) ]
13     # On créé la liste des sommets
14     S = [ i for i in range(n) ]
15     # Puis on trie simultanément la liste des sommets
16     # dans le même ordre que celle des degrés
17     ...
18     return S

```

6) Compléter alors la version gloutonne de la coloration.

```

1  def glouton(G):
2      n=len(G)
3      c = [0 for i in range(n)]
4      sommets = TriDegre(G)
5      ...
6      return c

```

7) Tester avec le graphe $G = [[0,0,1,1,0], [0,0,0,1,1], [1,0,0,1,1], [1,1,1,0,1], [0,1,1,1,0]]$.

8) Le nombre de couleurs utilisées est-il toujours minimal?

3 ALGORITHME DSATUR

Si s est un sommet du graphe G on appelle saturation du sommet s le nombre de couleurs (on ne compte pas les sommets non colorés) différentes dans les sommets voisins de s .

Par exemple pour le graphe :

Mat = $[[0,1,0,0,1,1], [1,0,1,0,0,0], [0,1,0,0,1,0], [0,0,0,0,1,0], [1,0,1,1,0,1], [1,0,0,0,1,0]]$
 de couleurs couleurs = $[1,2,1,3,4,2]$

Les saturations sont :

sommet	0	1	2	3	4	5
saturation	2	1	2	1	3	2

L'algorithme dsatur suit le processus suivant :

- On trie les sommets de G par degrés décroissants, on commence par le sommet de plus haut degré
- Tant que tout les sommets ne sont pas coloriés, on choisit un sommet non colorié de saturation maximale (et de plus haut degré en cas d'égalité), on le colorie avec la première couleur non utilisée par ses voisins

Bien comprendre : la saturation des sommets évolue au fur et à mesure du coloriage.

- 9) Créer une fonction unique(L) qui prend en argument une liste L , et retourne le nombre d'éléments distincts de L .
- 10) Compléter le code pour dsatur avec la trame donnée.

```

1  def dsatur(G):
2      n = len(G)
3      c = n*[0]
4      #liste des sommets par ordre de degrés décroissants
5      #tant qu'il y a des sommets sans couleur
6          #recherche du sommet à colorer, de degré de saturation maximum
7          #si le sommet i est sans couleur
8              #calcul du degré de saturation du sommet i
9              #le sommet i est-il de degré de saturation maximum ?
10         #choix de la plus petite couleur non présente chez les voisins du sommet choisi
11     return c

```

11) Le nombre de couleurs utilisées est-il toujours minimal?